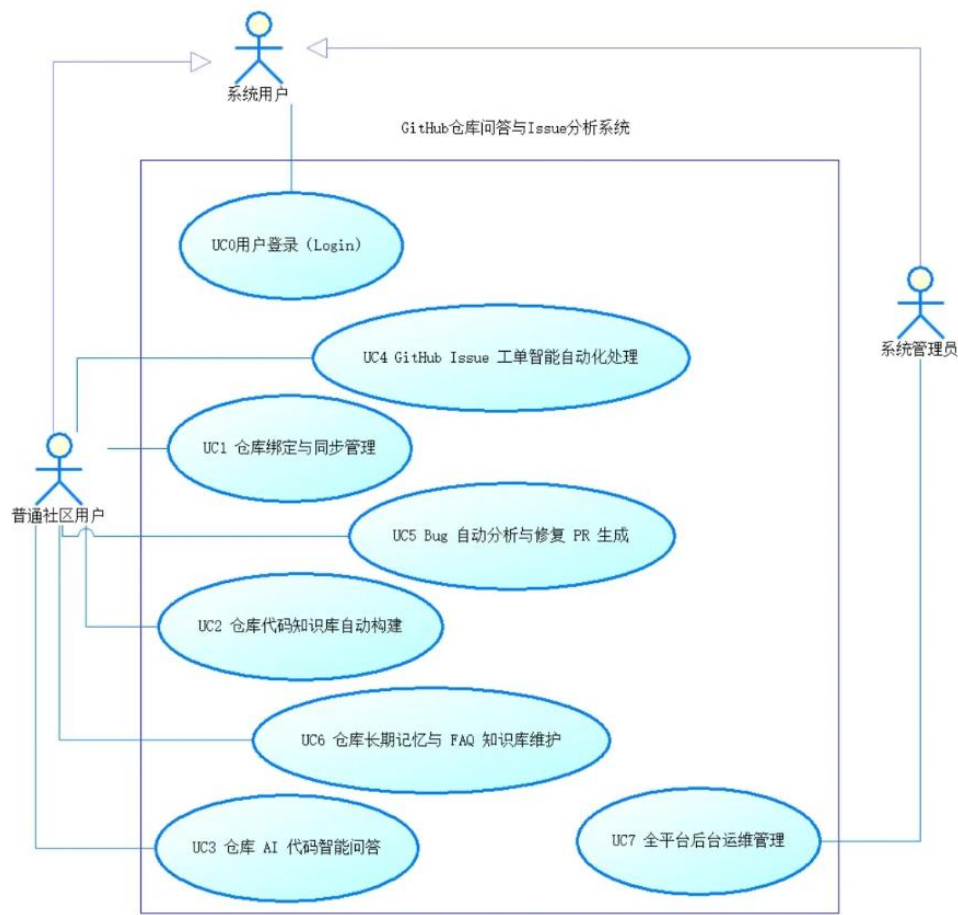


1. 用例模型

1.1 系统用例图说明



本系统（GitHub 仓库问答与 Issue 分析系统）划分为三个核心参与者：

- **系统用户**：抽象基类/泛化角色，代表所有登录系统的用户（UC0 用户登录）。
- **普通社区用户**：继承自系统用户，负责具体的仓库管理、知识库构建、AI 问答、Issue 处理与 Bug 修复（UC1~UC6）。
- **系统管理员**：继承自系统用户，负责全局后台运维与监控（UC7 全平台后台运维管理）。

1.2 核心用例列表

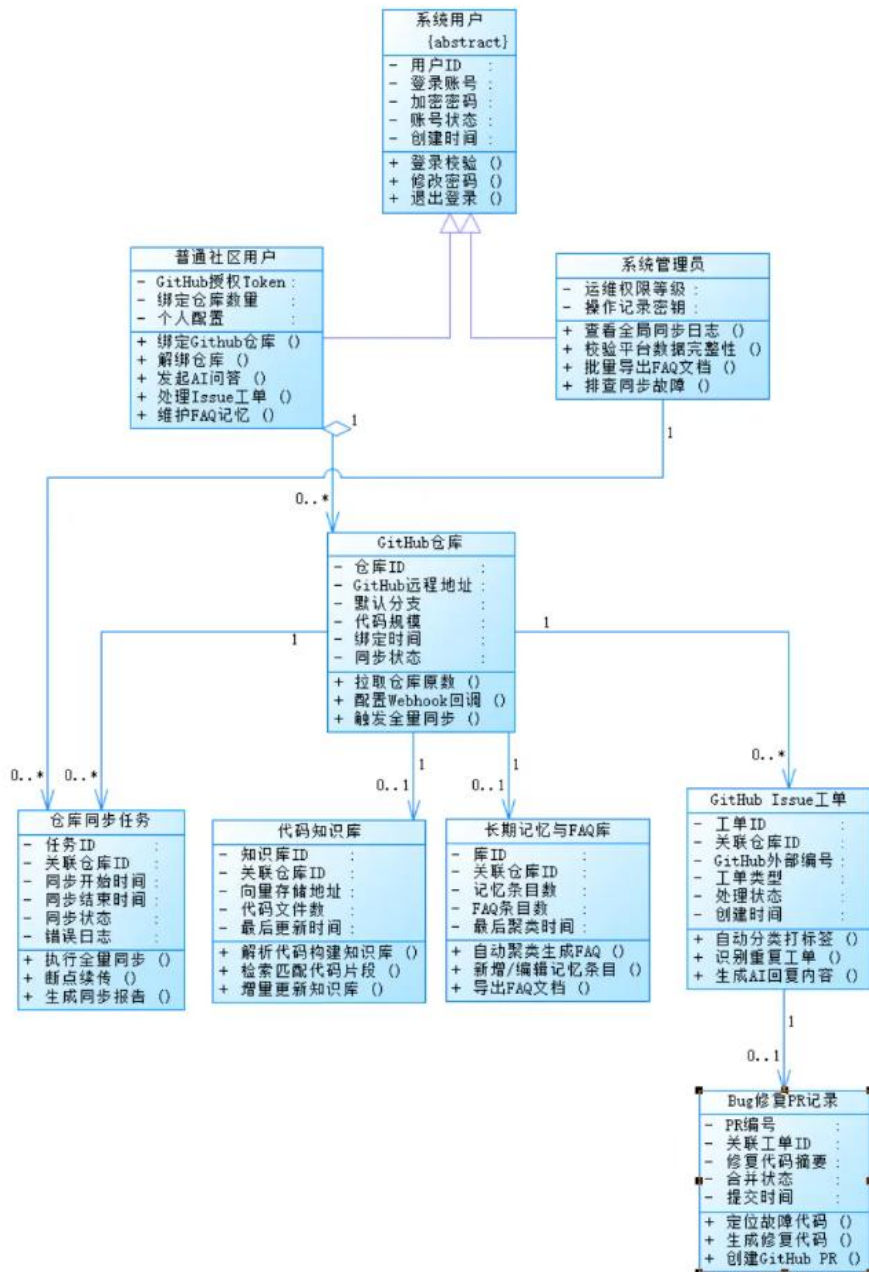
用例编号	用例名称	对应参与者	业务简述

UC0	用户登录	系统用户	用户身份认证与权限校验。
UC1	仓库绑定与同步管理	普通社区用户	绑定 GitHub 目标仓库，拉取/同步代码、Issue 与 PR 数据。
UC2	仓库代码知识库自动构建	普通社区用户	对仓库代码及文档进行切片与向量化索引构建。
UC3	仓库 AI 代码智能问答	普通社区用户	基于 RAG 技术对仓库代码与逻辑进行提问与解答。
UC4	GitHub Issue 工单智能自动化处理	普通社区用户	自动打标签、分类及自动回复/指派。
UC5	Bug 自动分析与修复 PR 生成	普通社区用户	定位报错根因，自动生成修复方案及代码补丁（PR）。
UC6	仓库长期记忆与 FAQ 知识库维护	普通社区用户	积累高频问答，维护历史上下文与 FAQ 知识库。
UC7	全平台后台运维管理	系统管理员	监控 Token 消耗、系统日志、向量库状态与用户权限管理。

2. 分析模型

分析模型从业务领域视角出发，定义系统内部的业务概念与行为（不引入具体编程语言和框架）。

2.1 概念类图



1.用户与权限模块

- 系统用户 {abstract}**: 抽象基类，封装通用用户属性（用户 ID、账号、加密密码、状态、创建时间）及基础认证方法（登录检验、修改密码、退出登录）。
- 普通社区用户**: 继承自系统用户，代表系统的主要使用群体。包含 GitHub Token 授权及绑定仓库指标，负责仓库绑定/解绑、AI 问答交互、Issue 工单处理及 FAQ 维护。
- 系统管理员**: 继承自系统用户，负责平台的全局运维。可查看全局同步日志、校验平台数据完整性、批量导出 FAQ 文档及排查同步故障。

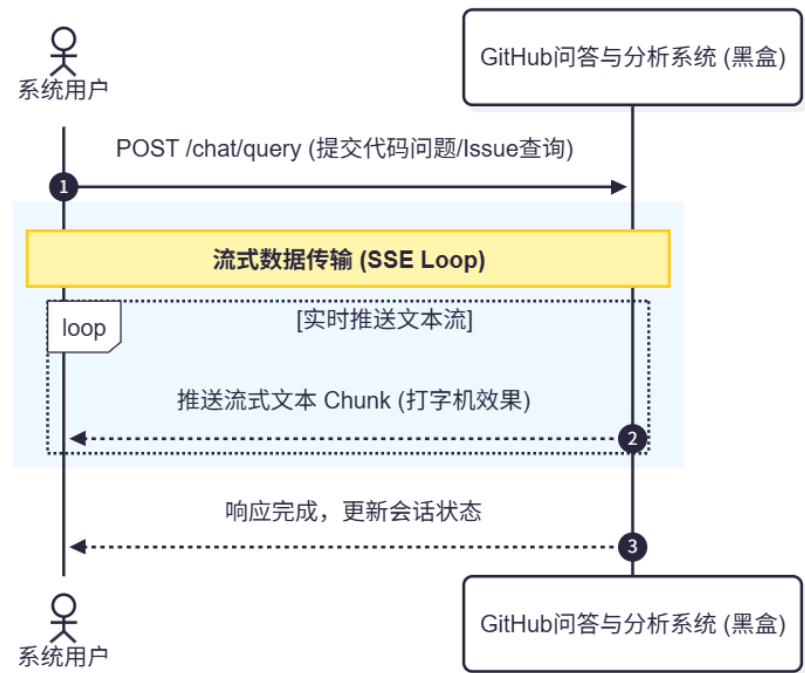
2.核心业务实体与知识库模块

- **GitHub 仓库**：系统的核心领域对象（1 个用户可管理 0.. * 个仓库）。存储远程地址、默认分支、代码规模等信息，提供拉取代码原数、配置 Webhook 及触发同步的方法。
- **仓库同步任务**：与 GitHub 仓库形成 1:0.. * 的关联，记录每次代码与 Issue 同步的执行状态、错误日志，支持断点续传与生成同步报告。
- **代码知识库**：与 GitHub 仓库构成 1:0..1 关系，负责解析代码结构、管理向量存储地址，并提供基于 semantic search 的代码匹配与增量更新能力。
- **长期记忆与 FAQ 库**：与 GitHub 仓库构成 1:0..1 关系，存储提炼后的记忆条目与 FAQ，支持自动聚类生成 FAQ 及文档导出。

3.智能工单与 Bug 修复 Agent 模块

- **GitHub Issue 工单**：与 GitHub 仓库构成 1:0.. * 关系，存储外部 Issue 编号、工单类型及状态，具备自动打标签、重复工单识别及 AI 回复生成能力。
- **Bug 修复 PR 记录**：与 Issue 工单形成 1:0..1 关联，记录修复代码摘要、合并状态等，负责定位故障代码、自动生成修复补丁并向 GitHub 提交 PR。

2.2 系统时序图

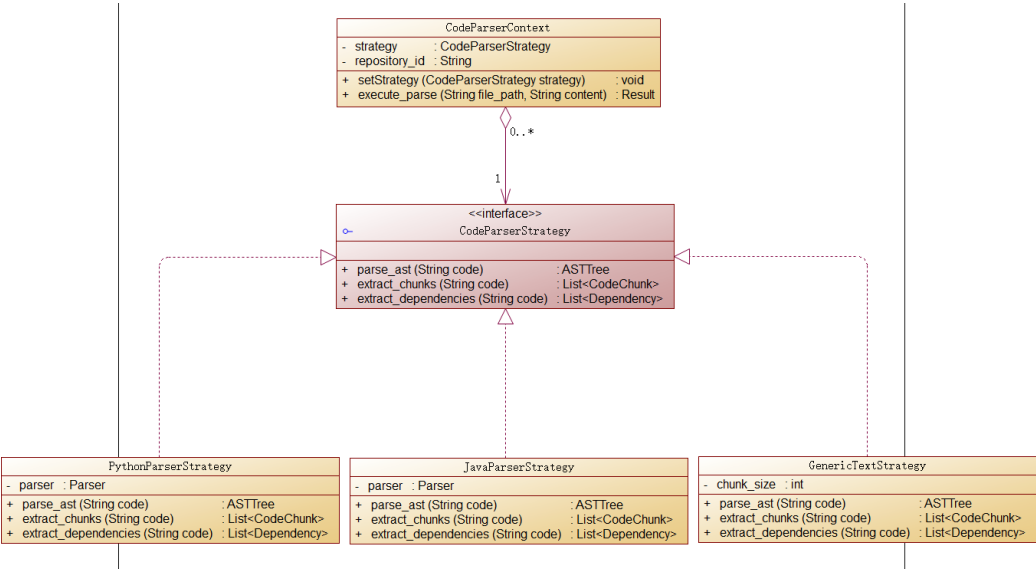


3. 设计模型

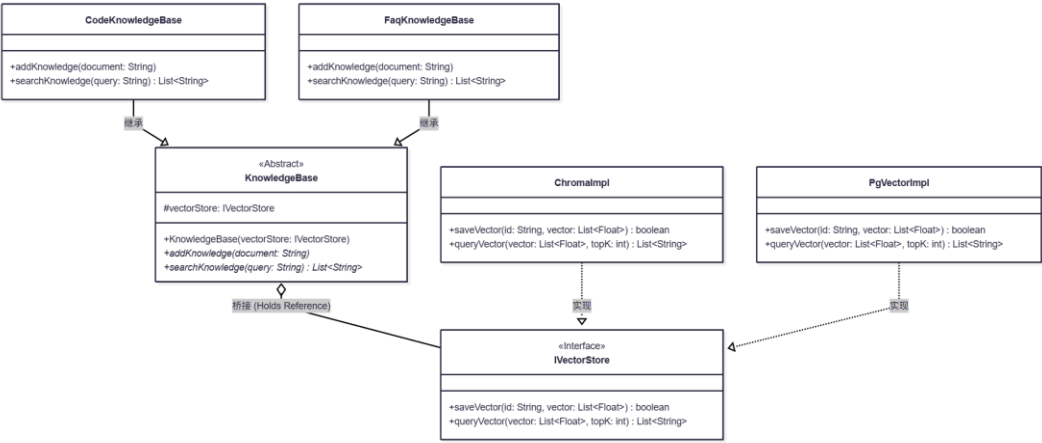
3.1 核心设计模式应用

为了实现灵活的大模型与存储扩展，本系统采用以下设计模式：

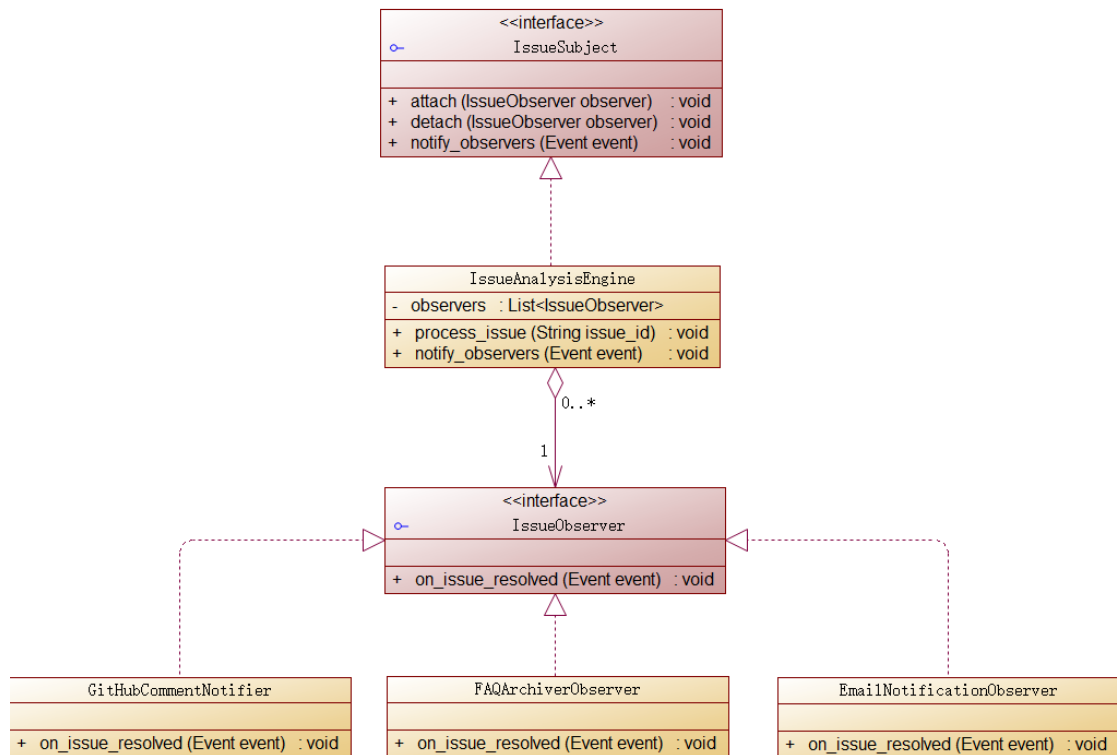
- 1. **策略模式**：解耦 LLM 模型实现，定义 ILlmService 接口，支持 OpenAiService、QwenService、OllamaService 的动态切换。



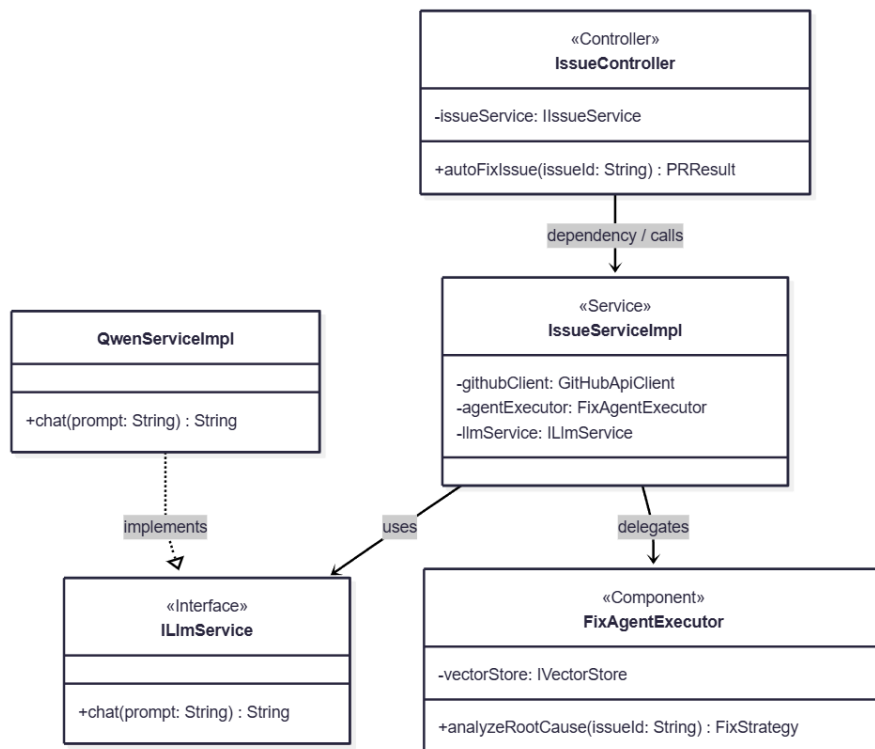
- 2. **桥接模式**：解耦知识库抽象与底层向量存储（VectorStoreImpl，如 ChromaImpl / PgVectorImpl）。



- 3. **观察者模式**：将 Issue 分析主引擎与后置通知/归档业务解耦，建立一对多的依赖关系。



3.2 详细设计类图



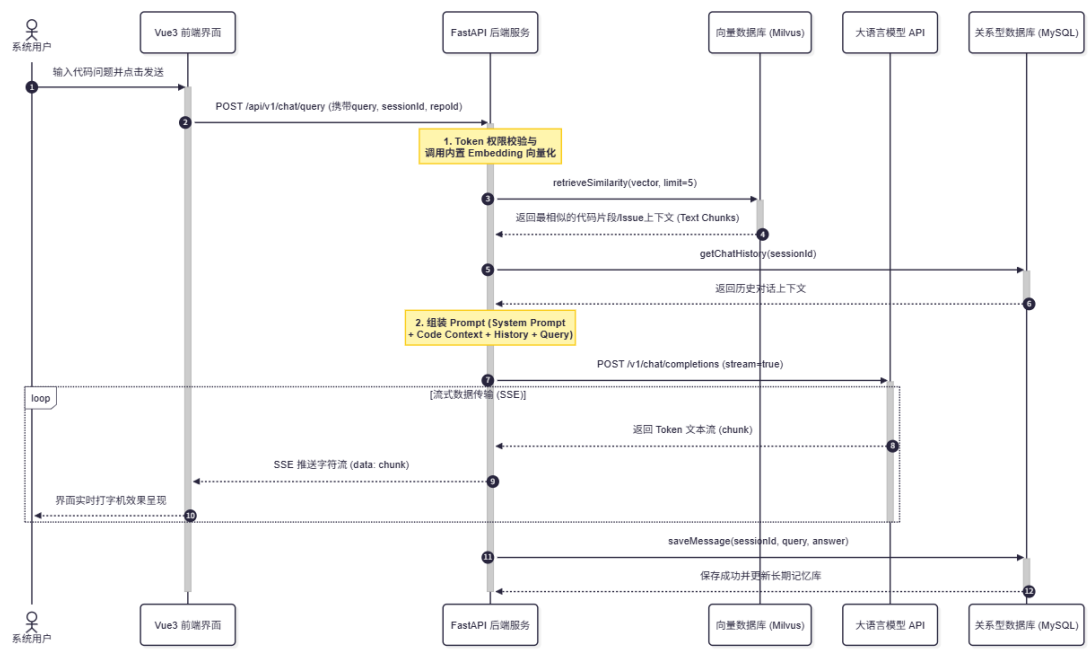
1) 架构设计与职责划分

- **IssueController (<>):** 控制层接口，接收前端/外部 Trigger 发起的 Issue 修复请求，暴露 autoFixIssue API。
- **IssueServiceImpl (<>):** 核心业务服务类，调度 GitHub API 客户端、Agent 执行器与大模型服务，完成工单处理全流程。
- **FixAgentExecutor (<>):** Bug 自动分析 Agent 执行引擎，依赖向量存储 (IVectorStore) 进行上下文定位与根因分析。
- **ILlmService (<>):** 大模型通用服务接口，解耦具体模型实现，提供统一的 chat 推理抽象。
- **QwenServiceImpl:** 大模型服务实现类，实现 ILlmService 接口，基于通义千问 (Qwen) 驱动 Agent 思考与代码生成。

2) 设计模式与依赖关系

1. **依赖注入与面向接口编程:** IssueServiceImpl 依赖抽象接口 ILlmService 而非具体类，便于后续无缝切换模型（如 OpenAI、DeepSeek）。
2. **职责单一与 Agent 解耦:** 将复杂的代码分析逻辑交由 FixAgentExecutor 组件封装，Controller 与 Service 仅做流程编排。

3.3 详细设计时序图



1. 流程概述

本设计时序图展现“UC3 仓库 AI 代码智能问答”用例的完整技术实现链路。

2.参与对象/生命线定义

- Vue3 前端界面：负责用户交互、捕获输入及通过 SSE 实时渲染打字机动画。
- FastAPI 后端服务：系统核心 Controller 与 Agent 引擎，负责权限校验、向量化、Prompt 组装与流式中继。
- 向量数据库 (Milvus)：存储仓库代码与 Issue 切片的向量索引，提供相似度检索能力。
- 大语言模型 API：提供 LLM 推理与流式 Token 生成能力。
- 关系型数据库 (MySQL)：持久化存储用户会话历史 (Session History) 与长期对话上下文。

3.详细交互步骤

请求发起与向量化

- 用户在 **Vue3 前端** 输入问题并发送，前端调用 POST `/api/v1/chat/query`。
- **FastAPI 后端** 完成鉴权后，调用 Embedding 模型将问题向量化。

多源上下文检索

- **向量检索**：后端向 **Milvus** 检索 Top-5 相似代码片段与 Issue 切片。
- **历史检索**：后端向 **MySQL** 读取该会话的历史对话记录。

Prompt 组装与流式调用

- 后端将 System Prompt、代码上下文、历史对话及当前问题拼装为复合 Prompt。
- 后端向 **大模型 API** 发起流式调用 (`stream=true`)。

SSE 实时流式响应

- 大模型逐 Token 返回文本流 (chunk)。
- 后端将 chunk 封装为 SSE 格式推送至前端，前端实时渲染打字机效果。

状态持久化

- 传输结束后，后端将问答记录异步保存至 **MySQL**，更新会话历史。